A Unified Structural and Operator Framework for the Linear n-Mass Pendulum

Yonatan Ali Rodríguez Arias*[1] and Sandra Arellano González†[1]

[1]Escuela Superior de Ingeniería Mecánica y Eléctrica Unidad Profesional "Adolfo López Mateos" Building 5, 3rd floor, Zacatenco, Delegación Gustavo A. Madero, Ciudad de México, 07738

May 2026

Abstract

A computational framework is presented that bridges the symbolic generation and linear dynamic propagation of the n-mass pendulum. The core contribution is the identification of a structural cumulative mass-sum rule that naturally governs the inertial, coupling, and gravitational terms derived from Lagrange's equations. This universal organizing principle eliminates the need for manual or recursive indexing, yielding a compact, explicit matrix assembly algorithm for arbitrary n.

The assembled linear system is solved both by standard modal decoupling and in state-space form. The latter is propagated with a scaled-and-squared truncated Peano–Baker (Taylor) series for the matrix exponential. The framework is intentionally limited to the linear regime, where both implementations agree with a reference matrix exponential to double-precision round-off. Because the transition matrix is constant for fixed parameters and timestep, it can be precomputed once and reused efficiently across repeated simulations, parameter sweeps, or control evaluations.

A brief discussion records the variation-of-constants representation for a possible nonlinear extension and points to modern exponential integrators; a full nonlinear method is outside the scope of this work. The proposed pipeline offers pedagogical transparency and explicit structural mapping, serving as a unified bridge between symbolic multibody generation and linear solution strategies.

Keywords: Multi-body Dynamics, n-mass Pendulum, Structural Generation, Linear State-Space, Matrix Exponential, Peano-Baker Series

*yonatanorion_84@hotmail.com

†ag.sandra012@gmail.com

1 Introduction

The pendulum remains one of the most fundamental systems in classical mechanics and continues to serve as a canonical model in multibody dynamics, robotics, vibration analysis, and control systems [1, 2, 3, 4, 5, 6]. While the equations of motion for single and double pendulum systems are widely known, explicitly deriving the equations governing an arbitrary n-mass pendulum rapidly becomes cumbersome due to the combinatorial growth of inertial and trigonometric coupling terms.

1.1 Related Work and Framework Positioning

Several mature approaches exist for multibody equation generation and simulation. Featherstone's articulated-body algorithm [7] prioritizes recursive computational efficiency, achieving $\mathcal{O}(n)$ scaling suitable for real-time applications, though it operates implicitly through factorized spatial algebra. Kane's method [8] provides a highly systematic approach for deriving minimal-order equations but typically requires manual formulation for specific topologies. General recursive formulations covering serial and tree-topology systems are treated in [9, 10]. Furthermore, general-purpose symbolic environments (e.g., SymPy mechanics) can derive explicit equations but often yield massive, unstructured expressions for large n.

The present work pursues a different and complementary objective: **explicit structural transparency**. Instead of focusing on raw computational speed for massive degrees of freedom, the main contribution is the identification of a cumulative mass-sum pattern that acts as a universal algebraic organizing principle throughout the linearized equations of motion.

The resulting symmetric positive-definite second-order system admits the standard generalized-eigenvalue (modal) solution. We use that solution as an independent reference and retain a state-space matrix-exponential implementation for applications in which a transition operator is convenient, such as control and repeated propagation. The matrix exponential is evaluated by a scaled-and-squared truncated Peano–Baker (Taylor) series; this numerical device is standard and is not claimed as a new matrix-exponential algorithm. The conceptual pipeline is illustrated in Figure 1.

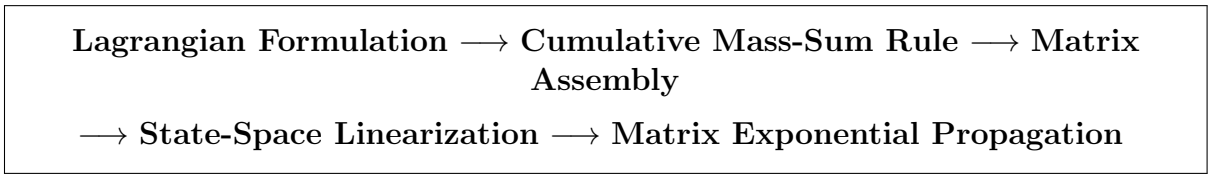

Figure 1: Conceptual pipeline of the proposed framework, bridging explicit structural generation and operator-based linear propagation.

The paper is organized as follows. Sections 2–4 develop the symbolic generation framework and introduce the cumulative mass-sum rule. Sections 5–6 derive the state-space system and the operator-based propagator. Section 7 presents validation results in the linear regime.

Section 8 briefly outlines a possible nonlinear extension (as a proof of concept) and delimits it as future work. Finally, Section 9 concludes.

2 Lagrangian Formulation of the n-Mass Pendulum

2.1 Equations of Motion

The dynamics of the system are derived from the Euler-Lagrange equations

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_j}\right) - \frac{\partial L}{\partial q_j} = 0, \tag{1}$$

where the Lagrangian is defined as $L = T - U$, with T the kinetic energy and U the potential energy. For the n-mass pendulum,

$$T = \frac{1}{2}\sum_{i=1}^{n} m_i v_i^2, \qquad U = g\sum_{i=1}^{n} m_i y_i,$$

and $v_i^2 = \dot{x}_i^2 + \dot{y}_i^2$. The generalized coordinates are $q_j = \theta_j$, $j = 1, \ldots, n$.

2.2 Coordinate Convention

Each generalized coordinate θ_i is measured with respect to the downward vertical direction, as illustrated in Figure 2. The Cartesian coordinates of the i-th mass are therefore

$$x_i = \sum_{k=1}^{i} l_k \sin\theta_k, \tag{2}$$

$$y_i = -\sum_{k=1}^{i} l_k \cos\theta_k. \tag{3}$$

The origin is taken at the pivot, with the positive x-axis to the right and the positive y-axis upward. This convention ensures that when all $\theta_k = 0$ (vertical downward equilibrium), $y_i = -\sum_{k=1}^{i} l_k$ and $x_i = 0$.

2.3 Explicit Derivation of Kinetic Energy

Differentiating Eqs. (2) and (3) with respect to time gives the velocity components:

$$\dot{x}_i = \sum_{k=1}^{i} l_k \dot{\theta}_k \cos\theta_k, \tag{4}$$

$$\dot{y}_i = \sum_{k=1}^{i} l_k \dot{\theta}_k \sin\theta_k. \tag{5}$$

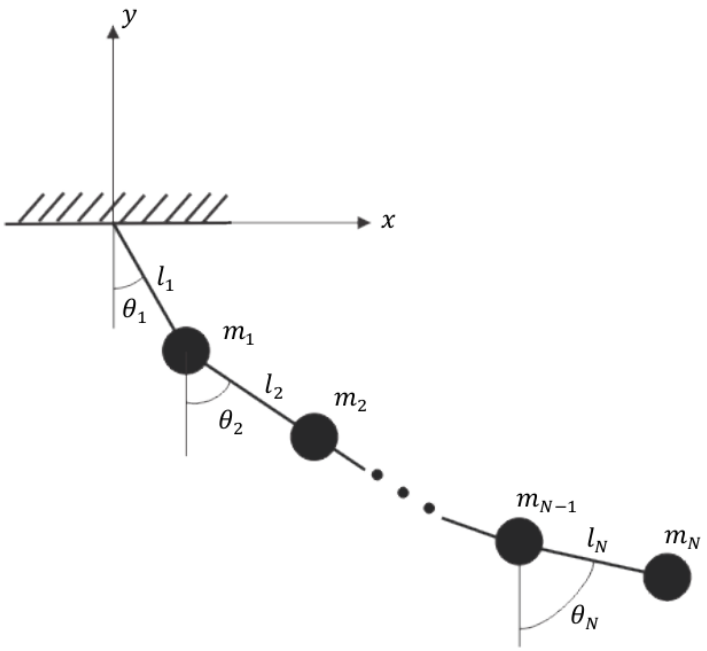

Figure 2: General n-mass pendulum with the absolute joint angle convention. Each angle θ_i is measured from the downward vertical. The coordinate origin is at the pivot.

Hence,

$$v_i^2 = \dot{x}_i^2 + \dot{y}_i^2 = \sum_{k=1}^{i}\sum_{r=1}^{i} l_k l_r \dot{\theta}_k \dot{\theta}_r \big(\cos\theta_k \cos\theta_r + \sin\theta_k \sin\theta_r\big). \tag{6}$$

Using the trigonometric identity $\cos\theta_k \cos\theta_r + \sin\theta_k \sin\theta_r = \cos(\theta_k - \theta_r)$, we obtain the compact expression

$$v_i^2 = \sum_{k=1}^{i}\sum_{r=1}^{i} l_k l_r \dot{\theta}_k \dot{\theta}_r \cos(\theta_k - \theta_r). \tag{7}$$

Note that the double sum runs over all pairs of links up to the i-th mass. The total kinetic energy is then

$$T = \frac{1}{2}\sum_{i=1}^{n} m_i \sum_{k=1}^{i}\sum_{r=1}^{i} l_k l_r \dot{\theta}_k \dot{\theta}_r \cos(\theta_k - \theta_r). \tag{8}$$

2.4 Potential Energy and its Linearization

Substituting y_i from Eq. (3) gives the potential energy

$$U = g\sum_{i=1}^{n} m_i \left(-\sum_{k=1}^{i} l_k \cos\theta_k\right) = -g\sum_{k=1}^{n}\left(\sum_{i=k}^{n} m_i\right) l_k \cos\theta_k. \tag{9}$$

For small oscillations we expand $\cos\theta_k \approx 1 - \theta_k^2/2$; the constant term does not affect the dynamics, so the relevant part is

$$U_{\text{quad}} = \frac{g}{2} \sum_{k=1}^{n} \left(\sum_{i=k}^{n} m_i \right) l_k \theta_k^2. \tag{10}$$

No cross terms $\theta_i \theta_j$ $(i \neq j)$ appear because the expansion of each cosine depends only on its own angle. This crucial simplification leads to a diagonal stiffness matrix, as shown in Section 5.3.

3 Structural Pattern of the Equations of Motion

For small systems, the equations of motion can be derived manually. However, the combinatorial growth of velocity products makes manual derivation impractical for large n. Inspection of systems with $n \in \{1, 2, 3, 4\}$ reveals a clear invariant structural property: every term in the d-th equation that involves coordinate θ_k or its derivatives contains the factor

$$\boxed{M_{d,k}^{\text{coeff}} = \sum_{i=\max(d,k)}^{n} m_i}. \tag{11}$$

3.1 The Cumulative Mass-Sum Rule as an Organizing Principle

This quantity represents the effective inertia dynamically shared by joints d and k. It acts as a universal organizing principle because the identical cumulative structure governs:

- diagonal and coupled acceleration terms,
- centrifugal and Coriolis-like velocity-squared terms,
- localized gravitational contributions.

Physical interpretation: when $k \geq d$, joint d contributes to the acceleration of all masses extending from coordinate k to the end of the chain. Conversely, when $k < d$, the inertial coupling relies on the deeper segment of the chain.

3.2 Explicit Example for $n = 4$

Applying the rule to the four-mass pendulum yields the first equation ($d = 1$), noting the proper dimensionality factor l_1:

$$\begin{aligned}
&(m_1 + m_2 + m_3 + m_4) l_1^2 \ddot{\theta}_1 + (m_2 + m_3 + m_4) l_1 l_2 \ddot{\theta}_2 \cos(\theta_1 - \theta_2) \\
&+ (m_3 + m_4) l_1 l_3 \ddot{\theta}_3 \cos(\theta_1 - \theta_3) + m_4 l_1 l_4 \ddot{\theta}_4 \cos(\theta_1 - \theta_4) \\
&+ (m_2 + m_3 + m_4) l_1 l_2 \dot{\theta}_2^2 \sin(\theta_1 - \theta_2) + (m_3 + m_4) l_1 l_3 \dot{\theta}_3^2 \sin(\theta_1 - \theta_3) \\
&+ m_4 l_1 l_4 \dot{\theta}_4^2 \sin(\theta_1 - \theta_4) + (m_1 + m_2 + m_3 + m_4) g l_1 \sin\theta_1 = 0
\end{aligned} \tag{12}$$

and the second equation ($d = 2$), scaled by l_2:

$$\begin{aligned}
&(m_2 + m_3 + m_4) l_2^2 \ddot{\theta}_2 + (m_3 + m_4) l_2 l_3 \ddot{\theta}_3 \cos(\theta_2 - \theta_3) + m_4 l_2 l_4 \ddot{\theta}_4 \cos(\theta_2 - \theta_4) \\
&+ (m_2 + m_3 + m_4) l_1 l_2 \ddot{\theta}_1 \cos(\theta_1 - \theta_2) + (m_3 + m_4) l_2 l_3 \dot{\theta}_3^2 \sin(\theta_2 - \theta_3) \\
&+ m_4 l_2 l_4 \dot{\theta}_4^2 \sin(\theta_2 - \theta_4) - (m_2 + m_3 + m_4) l_1 l_2 \dot{\theta}_1^2 \sin(\theta_1 - \theta_2) \\
&+ (m_2 + m_3 + m_4) g l_2 \sin\theta_2 = 0. \qquad (13)
\end{aligned}$$

The third and fourth equations can be obtained using the same methodology. Every coefficient is exactly a cumulative mass sum: $\sum_{i=\max(d,k)}^{n} m_i$ multiplied by the product of lengths $l_d l_k$.

4 Algorithmic Generation Framework

The complete symbolic structure can be mapped into a concise algorithm using the mass-sum rule, eliminating complex parsers or manual recursion.

4.1 Matrix Assembly from the Cumulative Sums

We define the mass matrix $\mathbf{M}(\boldsymbol{\theta})$ and the Coriolis/centrifugal vector $\mathbf{c}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})$ through the relation

$$\mathbf{M}(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}} + \mathbf{c}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) + \mathbf{g}(\boldsymbol{\theta}) = \mathbf{0},$$

where $\mathbf{g}(\boldsymbol{\theta})$ is the gravitational torque vector. The cumulative mass-sum rule gives explicit formulas:

$$M_{jk}(\boldsymbol{\theta}) = \left(\sum_{i=\max(j,k)}^{n} m_i \right) l_j l_k \cos(\theta_j - \theta_k), \qquad (14)$$

$$c_d(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) = \sum_{k=1}^{n} \left(\sum_{i=\max(d,k)}^{n} m_i \right) l_d l_k \, \dot{\theta}_k^2 \sin(\theta_d - \theta_k), \qquad (15)$$

$$g_d(\boldsymbol{\theta}) = \left(\sum_{i=d}^{n} m_i \right) g l_d \sin\theta_d. \qquad (16)$$

The sine difference alone determines the sign of every velocity-squared term; no additional index-dependent sign is present. For example, when $d = 2$ and $k = 1$, $\sin(\theta_2 - \theta_1) = -\sin(\theta_1 - \theta_2)$, exactly as in Eq. (13). For the linearized case (small angles) we set $\cos(\cdot) = 1$, $\sin(\cdot) = 0$ except in the gravitational term where $\sin\theta_d \approx \theta_d$.

4.2 Algorithm for Linear System Assembly

The following algorithm constructs the linearized system matrices directly from the cumulative sums without symbolic differentiation.

Algorithm 1 Linear Matrix Assembly via Cumulative Mass-Sum Rule

Require: masses $m_1, \ldots, m_n$, lengths $l_1, \ldots, l_n$, g
Ensure: mass matrix $\mathbf{M}$, stiffness matrix $\mathbf{K}$

$S_n \leftarrow m_n$
for $j \leftarrow n-1$ **down to** 1 **do**
    $S_j \leftarrow m_j + S_{j+1}$
end for
for $j \leftarrow 1$ **to** n **do**
    for $k \leftarrow 1$ **to** n **do**
        $M_{jk} \leftarrow S_{\max(j,k)} l_j l_k$
    end for
    $K_{jj} \leftarrow S_j g l_j$
end for

4.3 Algorithmic Scaling

The suffix masses S_j require $\mathcal{O}(n)$ work and are computed once. The subsequent loop assigns n^2 entries of $\mathbf{M}$ and n nonzero entries of $\mathbf{K}$, so the assembly requires $\mathcal{O}(n^2)$ work and storage. Recomputing the sum $\sum_{i=\max(j,k)}^{n} m_i$ inside every matrix entry would instead require $\mathcal{O}(n^3)$ work and is deliberately avoided.

Table 1: Number of entries assigned by the linear matrix assembly.

n	Entries of $\mathbf{M}$	Nonzero entries of $\mathbf{K}$	Total
2	4	2	6
4	16	4	20
6	36	6	42
8	64	8	72
10	100	10	110

5 Linearization and Matrix Assembly

5.1 Small-Angle Approximation

For small oscillations, we assume $|\theta_j| \ll 1$ rad. The following approximations are applied:

$$\sin\theta_j \approx \theta_j, \quad \cos(\theta_i - \theta_j) \approx 1,$$

and velocity-squared terms (which are second order in small quantities) are neglected. Under these assumptions the equations of motion reduce to the linear system

$$\mathbf{M}\ddot{\boldsymbol{\theta}} + \mathbf{K}\boldsymbol{\theta} = \mathbf{0}, \tag{17}$$

where $\mathbf{M}$ and $\mathbf{K}$ are constant matrices.

5.2 Derivation of the Mass Matrix

Inserting the small-angle approximation $\cos(\theta_j - \theta_k) \approx 1$ into the general expression for $M_{jk}(\boldsymbol{\theta})$ yields the constant mass matrix

$$M_{jk} = \left(\sum_{i=\max(j,k)}^{n} m_i \right) l_j l_k. \tag{18}$$

Note that $M_{jk} = M_{kj}$ because $\max(j,k) = \max(k,j)$. The matrix is symmetric and positive definite for positive masses and lengths.

5.3 Diagonal Structure of the Gravitational Stiffness Matrix

From the potential energy expansion (Eq. (10)), the gravitational torque on joint d is

$$g_d = \frac{\partial U_{\text{quad}}}{\partial \theta_d} = g \left(\sum_{i=d}^{n} m_i \right) l_d \theta_d.$$

Therefore the stiffness matrix $\mathbf{K}$ is diagonal:

$$K_{jk} = \left(\sum_{i=j}^{n} m_i \right) g l_j \, \delta_{jk}. \tag{19}$$

No off-diagonal entries appear because the quadratic potential contains only θ_j^2 terms; the coordinate convention (angles measured from the downward vertical) eliminates $\theta_i \theta_j$ cross couplings.

5.4 Standard Modal Decoupling

Because $\mathbf{M}$ and $\mathbf{K}$ are symmetric positive definite for positive masses and lengths, the generalized eigenproblem admits an $\mathbf{M}$-orthonormal modal matrix $\mathbf{U}$:

$$\mathbf{K}\mathbf{U} = \mathbf{M}\mathbf{U}\boldsymbol{\Omega}^2, \qquad \mathbf{U}^{\mathsf{T}}\mathbf{M}\mathbf{U} = \mathbf{I}, \qquad \mathbf{U}^{\mathsf{T}}\mathbf{K}\mathbf{U} = \boldsymbol{\Omega}^2, \tag{20}$$

where $\boldsymbol{\Omega} = \operatorname{diag}(\omega_1, \ldots, \omega_n)$ and $\omega_j > 0$. With $\boldsymbol{\theta} = \mathbf{U}\mathbf{q}$, the system decouples into $\ddot{q}_j + \omega_j^2 q_j = 0$, with the exact solution

$$q_j(t) = q_j(0)\cos(\omega_j t) + \frac{\dot{q}_j(0)}{\omega_j} \sin(\omega_j t). \tag{21}$$

This is the natural direct solution for the unforced linear second-order problem and is used below as an independent numerical reference.

5.5 State-Space Representation

Define the state vector $\mathbf{R}(t) = (\boldsymbol{\theta}(t), \dot{\boldsymbol{\theta}}(t))^{\mathsf{T}} \in \mathbb{R}^{2n}$. The linearized system can then be written as

$$\dot{\mathbf{R}} = \mathcal{A}\mathbf{R}, \qquad \mathcal{A} = \begin{pmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{M}^{-1}\mathbf{K} & \mathbf{0} \end{pmatrix}. \tag{22}$$

The matrix $\mathcal{A}$ is constant, which allows the use of explicit operator-based propagators. This first-order form is not required for the modal solution; it is retained because a reusable transition matrix is convenient in state-space control, forcing, and estimation workflows.

6 Operator-Based Dynamic Propagation

6.1 Peano-Baker Series for the Transition Matrix

Because $\mathcal{A}$ is constant, the state transition matrix from 0 to t is the matrix exponential $\Phi(t, 0) = e^{\mathcal{A}t}$.

For the autonomous system considered here, the Peano–Baker series is exactly the Taylor series of the matrix exponential. A direct truncated Taylor series can be inaccurate when $\|\mathcal{A}\Delta t\|$ is large [16], so the implementation uses scaling and squaring.

A convenient implementation defines $\mathbf{X} = \mathcal{A}\Delta t$ and chooses $s = \max\{0, \lceil \log_2(\|\mathbf{X}\|_\infty/\eta)\rceil\}$ with $\eta = 1/2$. For $\mathbf{B} = \mathbf{X}/2^s$, the approximation is

$$\Phi(\Delta t) \approx \left(\sum_{k=0}^{N} \frac{\mathbf{B}^k}{k!} \right)^{2^s}. \tag{23}$$

Scaling controls the Taylor remainder before repeated squaring. With $N = 20$, the tested systems agree with both `scipy.linalg.expm` and the modal solution to double-precision round-off. This implementation is included for transparency; robust library routines based on established matrix-exponential algorithms should be preferred outside the tested parameter range.

6.2 Propagation Algorithm and Reusability

Once $\Phi(\Delta t)$ is precomputed (using the scaled series), the linear propagation over a time step is simply

$$\mathbf{R}(t + \Delta t) = \Phi(\Delta t)\mathbf{R}(t).$$

This requires only matrix-vector multiplications, making it highly efficient for repeated time stepping. Importantly, because $\Phi(\Delta t)$ is constant for fixed system parameters and time step, it can be precomputed once and reused across many simulations (e.g., parameter sweeps, Monte Carlo runs, or real-time control loops). This reusability is a key practical advantage of the operator-based formulation.

7 Numerical Validation (Linear Regime)

7.1 Linear Validation

To ensure a comprehensive evaluation and guarantee reproducibility, the linear framework was validated across four distinct test cases. In all scenarios, the gravitational acceleration was set to $g = 9.81$ m/s^2 and the systems were released from rest ($\dot{\boldsymbol{\theta}}(0) = \mathbf{0}$). The specific parameter sets–masses m (in kg), lengths l (in m), and initial angular displacements $\boldsymbol{\theta}(0)$ (in radians)–are defined as follows:

- $n = 2$ **(uniform):** $m_i = 1.0$, $l_i = 1.0$, and $\boldsymbol{\theta}(0) = [0.1, 0.05]^\mathsf{T}$.
- $n = 3$ **(uniform):** $m_i = 1.0$, $l_i = 1.0$, and $\boldsymbol{\theta}(0) = [0.1, 0.05, 0.02]^\mathsf{T}$.
- $n = 3$ **(non-uniform):** $m = [2.0, 1.5, 0.8]^\mathsf{T}$, $l = [1.2, 0.9, 0.6]^\mathsf{T}$, and $\boldsymbol{\theta}(0) = [0.1, 0.05, 0.02]^\mathsf{T}$.
- $n = 5$ **(uniform):** $m_i = 1.0$, $l_i = 1.0$, and $\boldsymbol{\theta}(0) = [0.1, 0.05, 0.02, 0.01, 0.005]^\mathsf{T}$.

These configurations provide a baseline for verifying the assembly and propagation across varying dimensionalities and inertial distributions. The scaled Peano–Baker series was truncated at $N = 20$ with a time step of $\Delta t = 0.01$ s. Reference solutions were generated both by direct matrix exponentiation (`scipy.linalg.expm`) and by the generalized-eigenvalue solution in Eq. (21).

Table 2 confirms that both independent solution paths agree with the reference matrix exponential at the level of double-precision round-off.

Table 2: Maximum absolute error in $\theta_1(t)$ over $t \in [0, 8]$ s (reference: `scipy.linalg.expm`).

System	**Parameters**	Δt (s)	**Scaled series**	**Modal solution**
$n = 2$	uniform	0.01	6.88×10^{-15}	9.71×10^{-16}
$n = 3$	uniform	0.01	2.75×10^{-15}	1.57×10^{-15}
$n = 3$	non-uniform	0.01	1.60×10^{-15}	8.19×10^{-16}
$n = 5$	uniform	0.01	9.28×10^{-15}	1.31×10^{-15}

7.2 Nonlinear Sign and Energy Check

Although the main numerical study is linear, the sign in the exact velocity vector was checked directly using the double-pendulum parameters $m_1 = m_2 = l_1 = l_2 = 1$, $g = 9.81$, $\theta_1(0) = 5°$, $\theta_2(0) = 0$, and zero initial velocities. The exact nonlinear equations were integrated over $0 \le t \le 6$ with DOP853 using relative and absolute tolerances 10^{-12} and 10^{-14}, respectively. Table 3 shows the maximum drift in the physical energy $T + U$. The corrected expression remains at round-off, whereas inserting the erroneous extra piecewise sign produces the reported 10^{-4}-level drift.

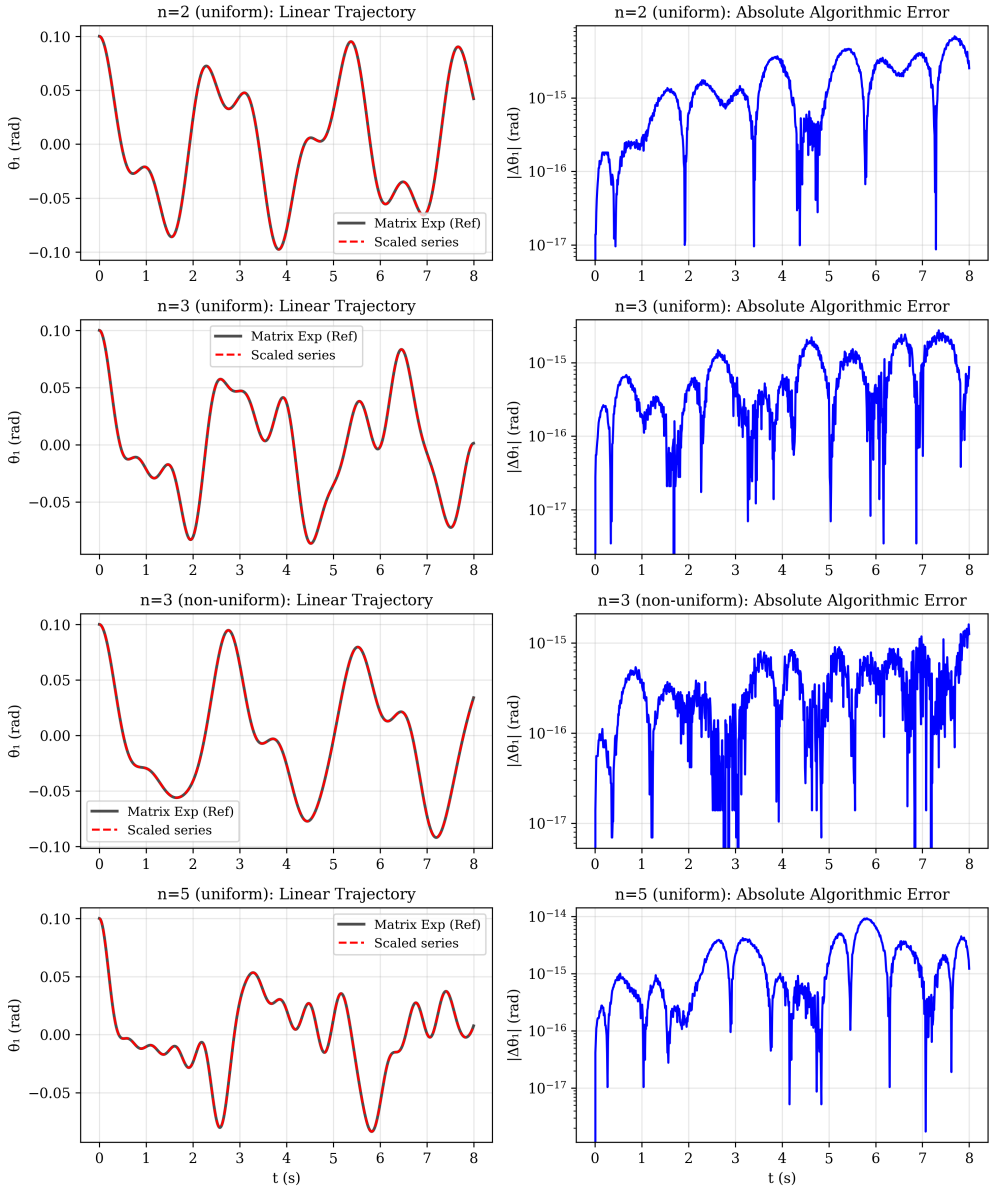

Figure 3: Linear validation results for the evaluated configurations. Left column: time-domain trajectories showing scaled-series stepping (dashed) versus the reference matrix exponential (solid). Right column: absolute algorithmic error confirming double-precision agreement.

Table 3: Maximum absolute energy drift in the nonlinear double-pendulum sign check.

Velocity-squared term	$\max_t \lvert E(t) - E(0) \rvert$
Corrected $\sin(\theta_d - \theta_k)$	3.55×10^{-15}
Former expression with extra piecewise sign	2.26×10^{-4}

7.3 Consistency Check with RK4

For completeness, the state-space implementation was also checked against classical fourth-order Runge–Kutta (RK4) on the same linear system. RK4 is a general-purpose time-stepper rather than a method tailored to this constant linear problem, so this comparison is only a discretization consistency check; the modal solution is the more appropriate direct baseline. Table 4 reports errors for a moderately large time step and for a ten-times-smaller RK4 step.

Table 4: Consistency check for the $n = 3$ non-uniform pendulum (linear regime).

Method	Δt (s)	**Max error (rad)**
Scaled series ($N = 20$)	0.1	9.62×10^{-15}
RK4	0.1	9.19×10^{-4}
RK4	0.01	9.42×10^{-8}

The scaled-series result demonstrates agreement with the matrix exponential after a fixed precomputation. It does not imply general superiority over Runge–Kutta methods, which are designed for a broader class of nonlinear and nonautonomous problems.

8 Discussion: Possible Nonlinear Extension as Future Work

Although the present work focuses on the linear regime, the variation-of-constants formula identifies how the same matrix-exponential kernel enters nonlinear exponential integrators. The following representation is included only to delimit a possible extension; it is not proposed as a competitive nonlinear solver. A rigorous implementation and comparison with modern exponential integrators [17] are left for future research.

8.1 Nonlinear Residual

The full nonlinear dynamics can be written as

$$\dot{\mathbf{R}} = \mathcal{A}\mathbf{R} + \mathbf{f}(\mathbf{R}), \tag{24}$$

where $\mathbf{f}(\mathbf{R})$ collects all terms omitted during linearization. Let $\mathbf{M}_0 = \mathbf{M}(\mathbf{0})$, so that the lower block of $\mathcal{A}\mathbf{R}$ is $-\mathbf{M}_0^{-1}\mathbf{K}\boldsymbol{\theta}$. The exact nonlinear acceleration is $-\mathbf{M}(\boldsymbol{\theta})^{-1}[\mathbf{c}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) + \mathbf{g}(\boldsymbol{\theta})]$; therefore

$$\mathbf{f}(\mathbf{R}) = \begin{pmatrix} \mathbf{0} \\ -\mathbf{M}(\boldsymbol{\theta})^{-1}\big[\mathbf{c}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) + \mathbf{g}(\boldsymbol{\theta})\big] + \mathbf{M}_0^{-1}\mathbf{K}\boldsymbol{\theta} \end{pmatrix}.$$

This expression includes both the nonlinear gravity and the angle dependence of the mass matrix, with the sign fixed by the original equation $\mathbf{M}(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}} + \mathbf{c} + \mathbf{g} = \mathbf{0}$. Importantly, the same cumulative mass-sum rule (Eq. 11) determines each coefficient in $\mathbf{f}$.

8.2 Picard Iteration Scheme

The corresponding nonlinear Volterra equation is

$$\mathbf{R}(t) = e^{\mathcal{A}t}\mathbf{R}(0) + \int_0^t e^{\mathcal{A}(t-s)}\mathbf{f}\big(\mathbf{R}(s)\big)\, ds. \tag{25}$$

Formal Picard iterations,

$$\mathbf{R}^{(m+1)}(t) = e^{\mathcal{A}t}\mathbf{R}(0) + \int_0^t e^{\mathcal{A}(t-s)}\mathbf{f}\big(\mathbf{R}^{(m)}(s)\big)\, ds, \tag{26}$$

starting from the linear solution $\mathbf{R}^{(0)}(t) = e^{\mathcal{A}t}\mathbf{R}(0)$, provide a local fixed-point construction. They are stated here to expose the integral structure, not as the recommended numerical method.

8.3 Limitations and Future Work

Picard iteration converges only locally under the usual contraction assumptions and, in this bare form, lacks adaptive step-size control and error estimation. Modern exponential integrators provide more appropriate discretizations of the variation-of-constants formula [17]. Their implementation and evaluation are beyond the present paper, which makes no numerical-performance claim for the nonlinear case.

9 Conclusions

This work presented a unified structural generation and solution framework for the *linear* n-mass pendulum. The core contribution is the explicit formulation and systematic use of the cumulative mass-sum rule (Eq. 11) as an organizing principle. Precomputing the suffix masses makes the linear matrix assembly genuinely quadratic. The standard generalized-eigenvalue solution and a scaled-and-squared matrix-exponential implementation agree with `scipy.linalg.expm` to double-precision round-off in the tested cases. Because the transition matrix is constant, it can be precomputed once and reused across many simulations.

The exact nonlinear sign was independently checked through energy conservation for the double pendulum. A complete nonlinear exponential integrator is left as future work. The framework's value lies in its pedagogical transparency and explicit structural mapping, making it suitable for educational purposes, control design, and parameter estimation tasks where explicit linear matrices are advantageous.

Code Availability

The complete Python implementation is available upon publication. For review purposes, the core code is provided in Appendix A.

The reference implementation and validation scripts are publicly archived at Zenodo: doi: 10.5281/zenodo.20102180

Licensing

A Core Python Implementation

```python
import numpy as np

def assemble_system(masses, lengths, g=9.81):
    """Assemble A, M, and K using precomputed suffix masses."""
    masses = np.asarray(masses, dtype=float)
    lengths = np.asarray(lengths, dtype=float)
    n = len(masses)
    S = np.cumsum(masses[::-1])[::-1]
    M = np.zeros((n, n))
    K = np.zeros((n, n))
    for j in range(n):
        for k in range(n):
            M[j, k] = S[max(j, k)] * lengths[j] * lengths[k]
        K[j, j] = S[j] * g * lengths[j]
    A = np.block([
        [np.zeros((n, n)), np.eye(n)],
        [-np.linalg.solve(M, K), np.zeros((n, n))]
    ])
    return A, M, K

def peano_baker_step(A, dt, N=20, eta=0.5):
    """Scaled-and-squared Taylor/Peano--Baker approximation."""
    B = A * dt
    norm_B = np.linalg.norm(B, ord=np.inf)
    s = 0 if norm_B <= eta else int(np.ceil(np.log2(norm_B / eta)))
    B = B / (2**s)
    dim = A.shape[0]
    Phi = np.eye(dim)
    term = np.eye(dim)
    for k in range(1, N + 1):
        term = (term @ B) / k
        Phi += term
    for _ in range(s):
        Phi = Phi @ Phi
    return Phi

def propagate_linear(Phi, R0, t):
    """Propagate linear system using precomputed transition matrix."""
    R = np.zeros((len(t), len(R0)))
    R[0] = R0
    for i in range(1, len(t)):
        R[i] = Phi @ R[i-1]
    return R
```

Listing 1: Core implementation of linear matrix assembly and propagation.

References

[1] Lobas, L., Generalized mathematical model of an inverted multilink pendulum with follower force, *International Applied Mechanics* **41**(5) (2005) 566–572. doi: 10.1007/s10778-005-0116-x

[2] Fritzkoski, P., Kaminski, H., Dynamics of a rope as a rigid multibody system, *Journal of Mechanics of Materials and Structures* **3**(6) (2008) 1059–1075. doi: 10.2140/jomms.2008.3.1059

[3] Fritzkoski, P., Kaminski, H., Dynamics of a rope modeled as a discrete system with extensible members, *Computational Mechanics* **44**(4) (2009) 473–480. doi: 10.1007/s00466-009-0382-6

[4] Pieranski, P., Tomaszewski, W., Dynamics of a rope and chains I: the fall of the folded chain, *New Journal of Physics* **7** (2005) 45. doi: 10.1088/1367-2630/7/1/045

[5] Kawaji, S., Kanazawa, K., Control of double inverted pendulum with elastic joint, *Proceedings of the IEEE/RSJ International Workshop on Intelligent Robots and Systems (IROS)*, Vol. 2 (1991) 946–951. doi: 10.1109/IROS.1991.174587

[6] Raymond, H., Lawrence, N., Pendulum model of a ponytail motion during walking and running, *Journal of Sound and Vibration* (2013). doi: 10.1016/j.jsv.2013.02.016

[7] Featherstone, R., *Rigid Body Dynamics Algorithms*, Springer, 2008. doi: 10.1007/978-1-4899-7560-7

[8] Kane, T., Levinson, D., *Dynamics: Theory and Applications*, McGraw-Hill, 1985.

[9] Jain, A., *Robot and Multibody Dynamics: Analysis and Algorithms*, Springer, 2010. doi: 10.1007/978-1-4419-7267-5

[10] Shabana, A.A., *Dynamics of Multibody Systems*, 4th ed., Cambridge University Press, 2010. doi: 10.1017/CBO9780511780521

[11] Blanes, S., Casas, F., Oteo, J., Ros, J., The Magnus expansion and some of its applications, *Physics Reports* **470**(5-6) (2009) 151–238. doi: 10.1016/j.physrep.2008.11.001

[12] Magnus, W., On the exponential solution of differential equations for a linear operator, *Communications on Pure and Applied Mathematics* **7**(4) (1954) 649–673. doi: 10.1002/cpa.3160070404

[13] Hairer, E., Nørsett, S.P., Wanner, G., *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer (1993). doi: 10.1007/978-3-540-78862-1

[14] Burton, T.A., *Volterra Integral and Differential Equations*, 2nd ed., Elsevier (2005). doi: 10.1016/S0076-5392(05)80001-5

[15] Atkinson, K., Han, W., *Theoretical Numerical Analysis*, 3rd ed., Springer (2009). doi: 10.1007/978-1-4419-0458-4

[16] Moler, C., Van Loan, C., Nineteen dubious ways to compute the exponential of a matrix, twenty-five years later, *SIAM Review* **45**(1) (2003) 3–49.

[17] Hochbruck, M., Ostermann, A., Exponential integrators, *Acta Numerica* **19** (2010) 209–286.